

# Arduino Cookbook : De la Théorie à la Pratique

## Chapitre 4 : Communication Série (Serial Communications)

---

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

## 4.0 Introduction : La Communication Série

### Le lien avec le monde extérieur

L'Arduino ne possède ni écran ni clavier. La communication série via le port USB est la méthode standard pour échanger des informations entre le microcontrôleur et l'ordinateur (ou d'autres appareils).

**Dans ce chapitre, nous allons apprendre à :**

- Envoyer des informations de débogage vers l'ordinateur.
- Formater des données complexes (Texte, Décimal, Hexadécimal).
- Recevoir et analyser des commandes envoyées depuis le PC.
- Comprendre la différence cruciale entre données ASCII et Binaires.
- Émuler un périphérique USB (Souris/Clavier) avec certaines cartes.

## 4.1 Envoyer des informations de débogage vers le PC

**Problème :** Vous voulez vérifier la valeur d'une variable ou voir où le programme en est.

**Solution :** Utiliser le Moniteur Série de l'IDE Arduino.

```
void setup() {  
    // Initialisation de la communication a 9600 bits par seconde (Bauds)  
    Serial.begin(9600);  
}  
  
void loop() {  
    int capteur = analogRead(A0);  
  
    Serial.print("Valeur du capteur : "); // Affiche sans sauter de ligne  
    Serial.println(capteur);             // Affiche la valeur et saute une ligne  
    delay(500);  
}
```

*Règle d'or : La vitesse configurée dans le `setup()` DOIT être identique à celle sélectionnée en bas à droite dans le Moniteur Série.*

## 4.2 Formater le texte et les nombres

**Problème** : Afficher des données dans un format spécifique (hexadécimal, binaire, ou avec un nombre précis de décimales).

**Solution** : Utiliser le deuxième paramètre optionnel de `Serial.print()`.

```
int valeur = 65;
Serial.println(valeur);           // Affiche "65" (Par défaut : base 10)
Serial.println(valeur, DEC);      // Affiche "65" (Decimal explicitement)
Serial.println(valeur, HEX);      // Affiche "41" (Hexadecimal)
Serial.println(valeur, OCT);      // Affiche "101" (Octal)
Serial.println(valeur, BIN);      // Affiche "1000001" (Binaire)

float temperature = 23.4567;
Serial.println(temperature, 1);   // Affiche "23.5" (Arrondi à 1 décimale)
```

**Figure 1:** Exemple de rendu dans le Moniteur Série (Fig 4-2)

## 4.3 Recevoir des données série sur l'Arduino

**Problème :** Comment faire pour que l'Arduino réagisse à une commande tapée sur l'ordinateur ?

**Solution :** Utiliser `Serial.available()` et `Serial.read()`.

### Le concept de Buffer (Mémoire Tampon)

L'Arduino possède un buffer de réception (64 octets). Quand l'ordinateur envoie des données, elles s'y stockent jusqu'à ce que votre programme les lise. `Serial.available()` renvoie le nombre d'octets en attente.

```
void loop() {  
    if (Serial.available() > 0) {           // S'il y a des données dans le buffer  
        char octetRecu = Serial.read();    // Lit le premier octet (et le retire du buffer)  
  
        if (octetRecu == 'A') {  
            digitalWrite(13, HIGH);        // Allume la LED si on reçoit 'A'  
        }  
    }  
}
```

## 4.4 Envoyer plusieurs champs de texte (CSV)

**Problème :** Vous avez plusieurs capteurs et vous voulez envoyer toutes leurs valeurs à un tableur (Excel) ou un programme externe.

**Solution :** Envoyer des valeurs séparées par des virgules (Format CSV).

```
void loop() {  
    int capteur1 = analogRead(A0);  
    int capteur2 = analogRead(A1);  
  
    Serial.print(capteur1); // Envoie la premiere valeur  
    Serial.print(",");      // Le separateur  
    Serial.println(capteur2); // Envoie la deuxieme valeur et saute la ligne  
  
    delay(100);  
}
```

*Le logiciel côté PC pourra facilement "découper" cette chaîne de caractères grâce à la virgule.*

## 4.5 Recevoir plusieurs valeurs simultanément (Parsing)

**Problème :** Vous envoyez une commande complexe comme "255,100" pour contrôler deux moteurs de manière asynchrone.

**Solution :** Utiliser la fonction `Serial.parseInt()`.

```
void loop() {  
  if (Serial.available() > 0) {  
    // parseInt() parcourt le texte reçu, s'arrête à la virgule,  
    // et convertit les caractères ASCII en entier (int)  
    int vitesseMoteur1 = Serial.parseInt();  
    int vitesseMoteur2 = Serial.parseInt();  
  
    // S'assurer de lire le retour à la ligne '\n' pour vider le buffer  
    if (Serial.read() == '\n') {  
      analogWrite(moteur1Pin, vitesseMoteur1);  
      analogWrite(moteur2Pin, vitesseMoteur2);  
    }  
  }  
}
```

## La confusion la plus courante (Margolis p.108)

**Texte ASCII** (`Serial.print`) : Si vous envoyez le nombre 123, l'Arduino envoie 3 octets : le caractère '1' (code 49), '2' (code 50) et '3' (code 51). C'est fait pour être lu par un humain dans le Moniteur Série.

**Données Binaires** (`Serial.write`) : Si vous envoyez la valeur 123 en binaire, l'Arduino n'envoie qu'un seul octet : 01111011. C'est illisible pour l'homme, mais très efficace pour communiquer avec d'autres logiciels ou machines.



## 4.6 Envoyer des données Binaires

**Problème :** Envoyer des informations compactes et rapides vers un logiciel (Processing, Python) sans conversion de texte.

**Solution :** Utiliser `Serial.write()`.

```
void loop() {  
    byte valeurPotentiometre = analogRead(A0) / 4; // Mise a l'echelle 0-255 (1 octet)  
  
    // N'affichera rien de comprehensible dans le Moniteur Serie !  
    // Mais enverra exactement un octet de donnee brute  
    Serial.write(valeurPotentiometre);  
  
    delay(50);  
}
```

**Figure 2:** Représentation visuelle des octets binaires vs ASCII

## 4.7 et 4.8 Interfaçage avec le logiciel Processing (PC)

L'Arduino travaille souvent en tandem avec **Processing** (un langage de programmation visuel pour PC) pour créer des interfaces graphiques.

### Code Processing (Côté Ordinateur) pour lire des données binaires :

```
import processing.serial.*;
Serial monPort;

void setup() {
    // Connexion au premier port serie de la liste a 9600 bauds
    monPort = new Serial(this, Serial.list()[0], 9600);
}

void draw() {
    if (monPort.available() > 0) {
        int valeurRecue = monPort.read(); // Lecture brute
        background(valeurRecue); // Change la couleur de fond selon l'Arduino
    }
}
```

## 4.9 Envoyer l'état de plusieurs broches efficacement

**Problème :** Envoyer l'état de 8 boutons sans saturer le port série avec du texte.

**Solution :** Compresser les données en utilisant les opérations binaires (*Bitwise*) vues au Chapitre 3.

```
void loop() {  
    byte etatBoutons = 0; // Un octet vide : 00000000  
  
    for (int i=0; i < 8; i++) {  
        // Lecture des broches 2 a 9  
        int etat = digitalRead(i + 2);  
        if (etat == HIGH) {  
            bitSet(etatBoutons, i); // Met le bit 'i' a 1  
        }  
    }  
  
    Serial.write(etatBoutons); // Envoie l'etat de 8 boutons en 1 seul octet !  
    delay(50);  
}
```

## 4.10 Émuler une Souris (Mouse Emulation)

**Problème :** Contrôler le curseur de l'ordinateur à partir d'un joystick connecté à l'Arduino.

**Solution :** Utiliser la bibliothèque Mouse.

### Restriction Matérielle

Cette fonctionnalité nécessite un Arduino avec un processeur USB natif, comme le **Leonardo** ou le **Micro** (ATmega32U4). Cela ne fonctionne pas directement sur l'Uno classique.

```
#include <Mouse.h>

void setup() {
    Mouse.begin(); // L'Arduino est maintenant reconnu comme une souris par le PC
}

void loop() {
    // Bouge le curseur de 10 pixels à droite, 5 en bas, molette 0
    Mouse.move(10, 5, 0);
    delay(1000);
}
```

## 4.11 Envoyer des commandes à Google Earth

**Application concrète du livre (Margolis) :** Utiliser l'Arduino pour naviguer dans des logiciels tiers.

**Méthode :** En utilisant un Leonardo, on peut émuler le clavier (bibliothèque `Keyboard.h`) pour envoyer des raccourcis clavier.

```
#include <Keyboard.h>

void setup() {
  pinMode(2, INPUT_PULLUP); // Bouton sur la broche 2
  Keyboard.begin();
}

void loop() {
  if (digitalRead(2) == LOW) {
    // Simule l'appui sur la fleche du haut
    Keyboard.press(KEY_UP_ARROW);
    delay(100);
    Keyboard.releaseAll(); // Tres important pour ne pas bloquer le clavier PC !
  }
}
```

# Dépannage : Le Moniteur Série affiche des caractères étranges

**Problème :** Au lieu du texte attendu, le Moniteur Série affiche des symboles incompréhensibles (ex : `ÿÿÿÃ¶`).

**Causes possibles :**

1. **Mauvais Baud Rate (99% des cas) :** La vitesse déclarée dans `Serial.begin(9600)` ne correspond pas au menu déroulant du Moniteur Série.
2. **Confusion ASCII/Binaire :** Votre programme utilise `Serial.write()` (qui envoie des octets invisibles ou spéciaux) au lieu de `Serial.print()` (qui envoie du texte formaté).
3. **Câblage partagé :** Vous avez branché un composant sur les broches 0 (RX) ou 1 (TX). Ces broches sont physiquement liées au convertisseur USB de l'Arduino Uno. Ne les utilisez pas si vous avez besoin du port série.

- **Ne pas inonder le buffer** : Mettez toujours un petit `delay()` dans votre `loop()` si vous envoyez des données en continu. L'Arduino est beaucoup plus rapide (16 MHz) que la vitesse de transfert USB à 9600 bauds.
- **Caractères de fin de ligne** : Lorsque vous utilisez `parseInt()` ou des protocoles personnalisés, configurez votre Moniteur Série sur "Nouvelle Ligne (NL) et Retour Chariot (CR)" (Newline & Carriage Return) pour délimiter vos commandes.
- **Mémoire RAM** : Si vous utilisez `Serial.print("Une très longue phrase");`, cette phrase est stockée dans la précieuse RAM. Utilisez la macro `F()` pour la forcer dans la mémoire Flash : `Serial.print(F("Une très longue phrase"));`.

## Résumé du Chapitre 4

- **Base de la communication** : `Serial.begin()` initialise, `Serial.print()` envoie du texte lisible, `Serial.write()` envoie de la donnée brute.
- **Écouteur vigilant** : Utilisez toujours `if(Serial.available() > 0)` avant de tenter une lecture avec `Serial.read()`.
- **Parsing natif** : `Serial.parseInt()` et `Serial.parseFloat()` simplifient considérablement l'extraction de nombres depuis une chaîne de texte.
- **Au-delà du texte** : Les cartes récentes (Leonardo, Micro) permettent d'aller plus loin en s'interfaçant comme de vrais périphériques informatiques (HID) via `Mouse.h` et `Keyboard.h`.

*Fin du Chapitre 4. Le Chapitre 5 abordera en détail les Entrées (Inputs) numériques et analogiques pour capturer les actions du monde physique.*